UNIVERSIDAD CATÓLICA ANDRÉS BELLO

EXTENSIÓN GUAYANA

FACULTAD INGENIERÍA

ESCUELA DE INFORMÁTICA

CÁTEDRA: TÓPICOS ESPECIALES DE PROGRAMACIÓN



PROYECTO 1

LOGIEXPRESS

MOTOR ANALÍTICO PARA SISTEMA DE GESTIÓN DE VUELOS

Profesora:

Noraimar Mora

Alumnos:

Andrés Boon 32.403.973

Jesus Valerio 32.015.059

Daniela Casanova 33.026.691

1. INTRODUCCIÓN

La empresa LogiExpress necesita gestionar una red compleja de rutas interurbanas donde la planificación de itinerarios exige procesar múltiples variables de forma simultánea. Determinar rutas válidas implica evaluar horarios, días de operación, tiempos de conexión y la optimización de costos financieros y temporales. Dada la alta complejidad combinatoria de estas redes, la solución requiere modelar los trayectos mediante grafos dirigidos y ponderados para explorar y optimizar eficientemente las alternativas de recorrido.

Para abordar problemas de esta magnitud, la elección de paradigmas de programación avanzados resulta clave. En particular, los enfoques lógico y funcional ofrecen mecanismos estructurados y expresivos para representar relaciones entre entidades, procesar estructuras de datos de manera recursiva y resolver sistemas regidos por restricciones estrictas.

Por ello, este proyecto tiene como objetivo desarrollar un motor analítico para procesar redes de vuelos y determinar itinerarios válidos mediante implementaciones contrastadas en ambos paradigmas. El sistema contempla la carga de datos, la gestión de conexiones y la búsqueda recursiva de rutas, permitiendo evaluar los costos de operación y comparar el desempeño de Prolog y Haskell frente a la complejidad del problema.

Link del repositorio: https://github.com/kirbyluka777/LogiExpress.git

2. MODELADO DE DATOS

2.1. Representación en Prolog

En Prolog, el dominio del problema se modela directamente como una base de conocimientos declarativa compuesta por hechos dinámicos y relaciones lógicas.

Cada conexión aérea se representa mediante un predicado de aridad 7:

```
vuelo(Origen, Destino, Salida, Llegada, Precio, Codigo, Dias).
```

Por ejemplo:

```
vuelo('Pto Ordaz', 'Maiquetia', '07:00', '08:00', 5600, 'Vlo#1',
'Diario').
```

La directiva `:- dynamic vuelo/7` permite la carga dinámica y actualización en tiempo de ejecución desde el archivo plano `vuelos.txt`.

El Tiempo Mínimo de Conexión (TMC) también se almacena como un hecho dinámico `:- dynamic tmc/1`, de modo permitiendo su modificación de manera interactiva mediante `tmc(minutos)` en tiempo de ejecución sin alterar la lógica de búsqueda.

2.2. Representación en Haskell

En Haskell, el vuelo se define mediante un tipo de dato con campos nombrados. Los tipos Ciudad, Minutos, Precio, CodigoVuelo y DiasOperacion son alias que documentan el significado de cada valor.

```
data Vuelo = Vuelo
    { origen  :: Ciudad
    , destino :: Ciudad
    , salida  :: Minutos
    , llegada :: Minutos
    , precio  :: Precio
    , codigo  :: CodigoVuelo
    , dias    :: DiasOperacion
    } deriving (Eq)
```

Grafo representa una lista de vuelos.

```
type Grafo = [Vuelo]
```

La lectura y validación están separadas en `LecturaDatos.hs`. El archivo se divide en líneas con `lines`, cada registro se separa por | y las horas se convierten a minutos desde medianoche. El precio se interpreta como Float. La función `parsearVuelo` verifica que existan siete campos, que los campos de texto requeridos no estén vacíos y que las horas respeten HH:MM. El resultado de la carga se representa con Either, de modo que un error de lectura o de parseo puede comunicarse al menú principal.

3. FUNCIONES Y PREDICADOS INVESTIGADOS

3.1 Investigación en Paradigma Lógico (Prolog)

- `findall/3`: Es un predicado nativo de Prolog que permite recopilar todas las soluciones posibles que satisfacen una meta o consulta dentro de la base de conocimientos mediante backtracking, almacenando los resultados en una lista

```
vuelos_llegada(Ciudad, Lista) :-
   findall(
       vuelo(O,D,S,L,C,Cod,Dia),
       (
           vuelo(O,D,S,L,C,Cod,Dia),
           ciudad_igual(D, Ciudad)
       ),
       Lista
   ).
```

- **Control de ciclos y listas de visitados con `[Cabeza|Cola]`:** Se implementó la manipulación estructural de listas utilizando el operador de descomposición `[Cabeza|Cola]`. Al recorrer recursivamente el grafo, la ciudad de destino actual se acumula en una lista auxiliar de "visitadas". Mediante un predicado auxiliar `no_visitada/2`, se verifica que ningún aeropuerto sea visitado más de una vez en el mismo itinerario, garantizando la terminación segura de la búsqueda.
- **Algoritmo de ordenamiento (Quicksort):** Para ver qué rutas son las más económicas o más rápidas se implementó el algoritmo de ordenamiento rápido `(Quicksort)` adaptado a estructuras de listas de rutas. El algoritmo utiliza predicados de partición que dividen la lista de itinerarios en subconjuntos menores y mayores respecto a un elemento pivote según el costo acumulado o el tiempo total de vuelo, ordenándolos recursivamente mediante la concatenación controlada `(append/3)`.

3.2 Investigación en Paradigma Declarativo (Haskell)

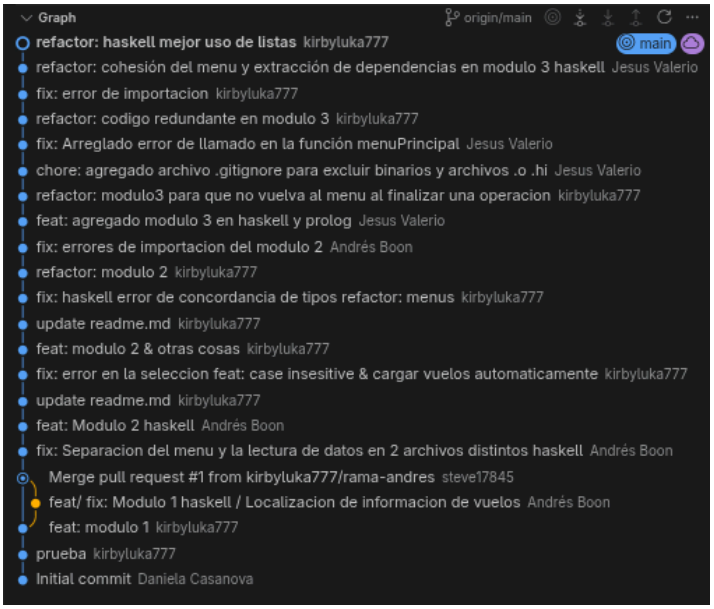
- **Funciones de Orden Superior y List Comprehensions:** Se hizo un uso extensivo de funciones como `map` (para iterar y calcular la suma de precios en una lista de vuelos) y `filter` (para descartar trayectos que exceden el límite de escalas o no operan en el día requerido). Asimismo, se emplearon list comprehensions (comprensiones de listas) para implementar de forma concisa y matemática la partición de elementos menores y mayores en el algoritmo `QuickSort`.

- **Recursión Explícita y Pattern Matching:** Dado que Haskell no posee un motor de búsqueda nativo, la exploración del grafo se implementó mediante un algoritmo recursivo explícito de `Búsqueda en Profundidad (DFS)`. El control de flujo y los casos base (como alcanzar el destino o procesar una lista vacía) se gestionaron exhaustivamente mediante Pattern Matching sobre la estructura de las listas `(x:xs)`.

- **Gestión del Estado con la Mónada IO:** Se investigó la separación arquitectónica de efectos secundarios. Todas las operaciones de interacción con el usuario (menús interactivos, `putStrLn`, `getLine`) y la manipulación del estado global (modificación del Tiempo Mínimo de Conexión) fueron estrictamente confinadas en funciones de

tipo `IO`. Esto permitió que la lógica central del Módulo 2 y Módulo 3 se mantuviera conformada exclusivamente por funciones puras.

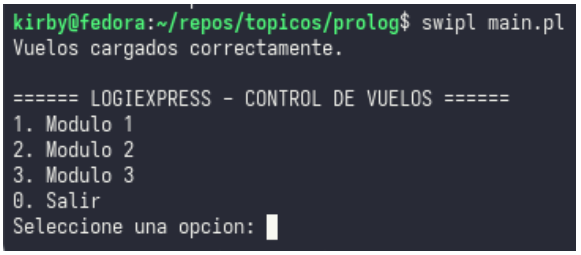
4. RESUMEN DE ACTIVIDADES Y RESPONSABLES

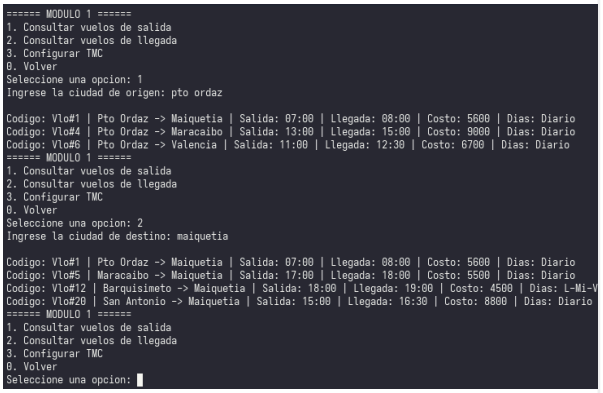


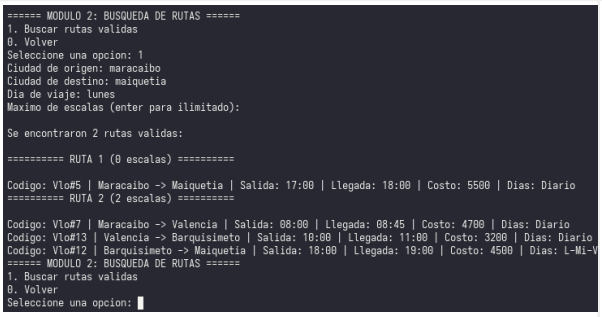
Integrante	Aporte
Daniela Casanova (kirbyluka777)	• Configuración y Documentación: Realizó la creación del proyecto base y mantuvo actualizada la documentación técnica (README.md). • Desarrollo Lógico: Participó en la implementación base del Módulo 1 y desarrolló las funcionalidades del Módulo 2. • Refactorización y Corrección de Errores: Solucionó errores de tipos en Haskell, optimizó el manejo y procesamiento de estructuras de datos (listas) para mejorar la eficiencia del código, corrigió problemas de importación y estabilizó la carga automática de vuelos. • Mejoras de Flujo y Navegación: Optimizó la estructura de los menús interactivos en los módulos para garantizar una navegación fluida, corrigiendo retornos automáticos no deseados al menú principal.
Andrés Boon (steve17845)	• Lectura y Procesamiento de Datos: Implementó la lógica fundamental del Módulo 1 en Haskell para la localización y extracción correcta de la información de los vuelos. • Arquitectura de Código: Diseñó la separación de responsabilidades en Haskell, dividiendo la lectura de datos y la gestión de menús en archivos distintos. • Desarrollo Lógico: Desarrolló la implementación base del Módulo 2 en Haskell. • Gestión de Integración: Estuvo a cargo de la resolución de conflictos de importación, integración de código mediante la fusión de ramas (Pull Requests) y estabilización de dependencias.
Jesús Valerio	• Desarrollo de Optimización: Implementó en su totalidad el Módulo 3 (Optimización y Filtrado de rutas) desarrollando las soluciones algorítmicas de manera simultánea para Haskell y Prolog. • Refactorización de Algoritmos: Mejoró la cohesión del código en el Módulo 3 de Haskell mediante la extracción de dependencias, la implementación de un ordenamiento eficiente (QuickSort) y la reestructuración del flujo del menú. • Corrección Crítica: Arregló errores de compilación asociados a las llamadas de funciones principales asegurando la correcta ejecución del sistema. • Mantenimiento del Repositorio: Configuró las reglas de exclusión del proyecto creando el archivo .gitignore para evitar la subida de archivos binarios e intermedios.

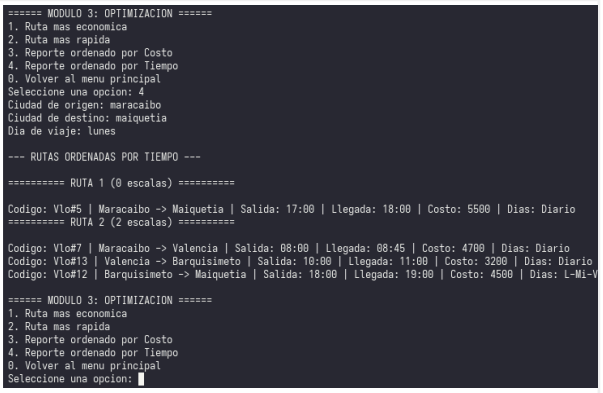
5. EVIDENCIAS DE EJECUCIÓN

Ejecución en Prolog









Ejecución en Haskell

Módulo 1:

```
====== LOGIEXPRESS - CONTROL DE VUELOS ======
1. Modulo 1 - Consultas basicas
2. Modulo 2 - Busqueda de rutas
3. Modulo 3: Optimizacion de Rutas
0. Salir
Seleccione una opcion: 1

====== MODULO 1 - CONSULTAS BASICAS ======
1. Consultar vuelos de salida
2. Consultar vuelos de llegada
3. Configurar TMC
0. Volver al menu principal
Seleccione una opcion: 1
Ingrese la ciudad de origen: PTO ordaz
Vlo#1: Pto Ordaz -> Maiquetia | Salida: 07:00 | Llegada: 08:00 | Precio: 5600.0 | Dias: Diario
Vlo#4: Pto Ordaz -> Maracaibo | Salida: 13:00 | Llegada: 15:00 | Precio: 9000.0 | Dias: Diario
Vlo#6: Pto Ordaz -> Valencia | Salida: 11:00 | Llegada: 12:30 | Precio: 6700.0 | Dias: Diario

====== MODULO 1 - CONSULTAS BASICAS ======
1. Consultar vuelos de salida
2. Consultar vuelos de llegada
3. Configurar TMC
0. Volver al menu principal
Seleccione una opcion: 2
Ingrese la ciudad de destino: Maiquetia
Vlo#1: Pto Ordaz -> Maiquetia | Salida: 07:00 | Llegada: 08:00 | Precio: 5600.0 | Dias: Diario
Vlo#5: Maracaibo -> Maiquetia | Salida: 17:00 | Llegada: 18:00 | Precio: 5500.0 | Dias: Diario
Vlo#12: Barquisimeto -> Maiquetia | Salida: 18:00 | Llegada: 19:00 | Precio: 4500.0 | Dias: L-Mi-V
Vlo#20: San Antonio -> Maiquetia | Salida: 15:00 | Llegada: 16:30 | Precio: 8800.0 | Dias: Diario

====== MODULO 1 - CONSULTAS BASICAS ======
1. Consultar vuelos de salida
2. Consultar vuelos de llegada
3. Configurar TMC
0. Volver al menu principal
Seleccione una opcion: 3
TMC actual: 45 minutos. Nuevo TMC: 50
TMC configurado en 50 minutos.

====== MODULO 1 - CONSULTAS BASICAS ======
1. Consultar vuelos de salida
2. Consultar vuelos de llegada
3. Configurar TMC
0. Volver al menu principal
Seleccione una opcion: 3
TMC actual: 50 minutos. Nuevo TMC:
```

Módulo 2:

```
====== MODULO 2 - BUSQUEDA DE RUTAS ======
1. Buscar rutas validas
0. Volver al menu principal
Seleccione una opcion: 1
Ciudad de origen: Pto ordaZ
Ciudad de destino: Barcelona
Dia de la semana (ej. Lunes): Martes
Numero maximo de escalas (K): 1

Se encontraron 1 rutas validas:
----------------------------------------
  * Vlo#1: Pto Ordaz -> Maiquetia | Salida: 07:00 | Llegada: 08:00 | Precio: 5600.0 | Dias: Diario
  * Vlo#14: Maiquetia -> Barcelona | Salida: 11:00 | Llegada: 11:50 | Precio: 4100.0 | Dias: Diario
  ==> Costo total del viaje: 9700.0
----------------------------------------

====== MODULO 2 - BUSQUEDA DE RUTAS ======
1. Buscar rutas validas
0. Volver al menu principal
Seleccione una opcion: 1
Ciudad de origen: San antonio
Ciudad de destino: Maiquetia
Dia de la semana (ej. Lunes): Miercoles
Numero maximo de escalas (K): 4

Se encontraron 2 rutas validas:
----------------------------------------
  * Vlo#18: San Antonio -> Maracaibo | Salida: 09:45 | Llegada: 10:45 | Precio: 6100.0 | Dias: L-Mi-V
  * Vlo#5: Maracaibo -> Maiquetia | Salida: 17:00 | Llegada: 18:00 | Precio: 5500.0 | Dias: Diario
  ==> Costo total del viaje: 11600.0
----------------------------------------
----------------------------------------
  * Vlo#20: San Antonio -> Maiquetia | Salida: 15:00 | Llegada: 16:30 | Precio: 8800.0 | Dias: Diario
  ==> Costo total del viaje: 8800.0
----------------------------------------
```

```
====== MODULO 2 - BUSQUEDA DE RUTAS ======
1. Buscar rutas validas
0. Volver al menu principal
Seleccione una opcion: 1
Ciudad de origen: Maiquetia
Ciudad de destino: Porlamar
Dia de la semana (ej. Lunes): Sabado
Numero maximo de escalas (K): 0

Se encontraron 0 rutas validas:
No hay rutas disponibles que cumplan las restricciones.

====== MODULO 2 - BUSQUEDA DE RUTAS ======
1. Buscar rutas validas
0. Volver al menu principal
Seleccione una opcion: 1
Ciudad de origen: Maiquetia
Ciudad de destino: Porlamar
Dia de la semana (ej. Lunes): Sabado
Numero maximo de escalas (K): 2

Se encontraron 1 rutas validas:
----------------------------------------
  * Vlo#14: Maiquetia -> Barcelona | Salida: 11:00 | Llegada: 11:50 | Precio: 4100.0 | Dias: Diario
  * Vlo#15: Barcelona -> Porlamar | Salida: 13:00 | Llegada: 13:40 | Precio: 3500.0 | Dias: Diario
  ==> Costo total del viaje: 7600.0
----------------------------------------
```

Módulo 3:

```
====== MODULO 3 - OPTIMIZACION DE RUTAS ======
1. Ruta mas economica (Costo Minimo)
2. Ruta mas rapida (Menor Tiempo Total)
3. Reporte ordenado por Costo
4. Reporte ordenado por Tiempo
0. Volver al menu principal
Seleccione una opcion: 1
Ciudad de origen: Pto ordaz
Ciudad de destino: porlamar
Dia de la semana: lunes

--- RUTA MAS ECONOMICA ---
----------------------------------------
  * Vlo#1: Pto Ordaz -> Maiquetia | Salida: 07:00 | Llegada: 08:00 | Precio: 5600.0 | Dias: Diario
  * Vlo#2: Maiquetia -> Porlamar | Salida: 09:00 | Llegada: 10:00 | Precio: 4800.0 | Dias: L-M-Mi-J-V
  ==> Costo total del viaje: 10400.0
----------------------------------------
```

```
====== MODULO 3 - OPTIMIZACION DE RUTAS ======
1. Ruta mas economica (Costo Minimo)
2. Ruta mas rapida (Menor Tiempo Total)
3. Reporte ordenado por Costo
4. Reporte ordenado por Tiempo
0. Volver al menu principal
Seleccione una opcion: 2
Ciudad de origen: Pto ordaz
Ciudad de destino: porlamar
Dia de la semana: Sabado

--- RUTA MAS RAPIDA ---
----------------------------------------
  * Vlo#1: Pto Ordaz -> Maiquetia | Salida: 07:00 | Llegada: 08:00 | Precio: 5600.0 | Dias: Diario
  * Vlo#14: Maiquetia -> Barcelona | Salida: 11:00 | Llegada: 11:50 | Precio: 4100.0 | Dias: Diario
  * Vlo#15: Barcelona -> Porlamar | Salida: 13:00 | Llegada: 13:40 | Precio: 3500.0 | Dias: Diario
  ==> Costo total del viaje: 13200.0
----------------------------------------
```

```
====== MODULO 3 - OPTIMIZACION DE RUTAS ======
1. Ruta mas economica (Costo Minimo)
2. Ruta mas rapida (Menor Tiempo Total)
3. Reporte ordenado por Costo
4. Reporte ordenado por Tiempo
0. Volver al menu principal
Seleccione una opcion: 3
Ciudad de origen: PTO ordaz
Ciudad de destino: PorlaMar
Dia de la semana: Sabado

--- RUTAS ORDENADAS POR COSTO ---
----------------------------------------
  * Vlo#6: Pto Ordaz -> Valencia | Salida: 11:00 | Llegada: 12:30 | Precio: 6700.0 | Dias: Diario
  * Vlo#9: Valencia -> Porlamar | Salida: 17:00 | Llegada: 18:00 | Precio: 4300.0 | Dias: L-S-D
  ==> Costo total del viaje: 11000.0
----------------------------------------
----------------------------------------
  * Vlo#1: Pto Ordaz -> Maiquetia | Salida: 07:00 | Llegada: 08:00 | Precio: 5600.0 | Dias: Diario
  * Vlo#14: Maiquetia -> Barcelona | Salida: 11:00 | Llegada: 11:50 | Precio: 4100.0 | Dias: Diario
  * Vlo#15: Barcelona -> Porlamar | Salida: 13:00 | Llegada: 13:40 | Precio: 3500.0 | Dias: Diario
  ==> Costo total del viaje: 13200.0
----------------------------------------
```

```
====== MODULO 3 - OPTIMIZACION DE RUTAS ======
1. Ruta mas economica (Costo Minimo)
2. Ruta mas rapida (Menor Tiempo Total)
3. Reporte ordenado por Costo
4. Reporte ordenado por Tiempo
0. Volver al menu principal
Seleccione una opcion: 4
Ciudad de origen: Maiquetia
Ciudad de destino: Valencia
Dia de la semana: Lunes
No hay rutas validas para optimizar.
```

```
====== MODULO 3 - OPTIMIZACION DE RUTAS ======
1. Ruta mas economica (Costo Minimo)
2. Ruta mas rapida (Menor Tiempo Total)
3. Reporte ordenado por Costo
4. Reporte ordenado por Tiempo
0. Volver al menu principal
Seleccione una opcion: 4
Ciudad de origen: Pto Ordaz
Ciudad de destino: Porlamar
Dia de la semana: lunes

--- RUTAS ORDENADAS POR TIEMPO ---
----------------------------------------
  * Vlo#1: Pto Ordaz -> Maiquetia | Salida: 07:00 | Llegada: 08:00 | Precio: 5600.0 | Dias: Diario
  * Vlo#2: Maiquetia -> Porlamar | Salida: 09:00 | Llegada: 10:00 | Precio: 4800.0 | Dias: L-M-Mi-J-V
  ==> Costo total del viaje: 10400.0
----------------------------------------
----------------------------------------
  * Vlo#1: Pto Ordaz -> Maiquetia | Salida: 07:00 | Llegada: 08:00 | Precio: 5600.0 | Dias: Diario
  * Vlo#14: Maiquetia -> Barcelona | Salida: 11:00 | Llegada: 11:50 | Precio: 4100.0 | Dias: Diario
  * Vlo#15: Barcelona -> Porlamar | Salida: 13:00 | Llegada: 13:40 | Precio: 3500.0 | Dias: Diario
  ==> Costo total del viaje: 13200.0
----------------------------------------
----------------------------------------
  * Vlo#6: Pto Ordaz -> Valencia | Salida: 11:00 | Llegada: 12:30 | Precio: 6700.0 | Dias: Diario
  * Vlo#9: Valencia -> Porlamar | Salida: 17:00 | Llegada: 18:00 | Precio: 4300.0 | Dias: L-S-D
  ==> Costo total del viaje: 11000.0
----------------------------------------
```

6. CONCLUSIONES

El proyecto logra modelar exitosamente la red aérea de LogiExpress como un grafo dirigido ponderado, separando la arquitectura del sistema en módulos de carga, consulta, búsqueda y optimización. Al implementar este motor analítico en dos paradigmas distintos, se pudo contrastar directamente cómo cada lenguaje gestiona la complejidad algorítmica, el estado y el desempeño frente al mismo problema.

En el paradigma lógico, Prolog aborda el problema mediante una base de conocimientos declarativa, donde los vuelos son hechos y las restricciones son relaciones. Su mayor ventaja competitiva radica en su expresividad y en el uso del backtracking nativo; herramientas como findall/3 permitieron delegar la exploración exhaustiva del grafo al motor de inferencia, agilizando enormemente el desarrollo de la búsqueda. Asimismo, el manejo del estado (como el Tiempo Mínimo de Conexión) se resolvió de forma inmediata mediante predicados dinámicos. Sin embargo, esta flexibilidad sacrifica la pureza declarativa al introducir mutabilidad global y exige un control manual riguroso (listas de visitados) para evitar que la unificación caiga en ciclos infinitos.

Por su parte, Haskell aborda el problema desde el paradigma funcional utilizando Tipos de Datos Algebraicos (registros) y procesamiento de listas inmutables. A diferencia de Prolog, Haskell exigió la construcción explícita del motor de búsqueda en profundidad (DFS) mediante recursividad, funciones de orden superior (filter, map) y concatenación. No obstante, Haskell compensó esta mayor verbosidad algorítmica con un desempeño superior en seguridad y estabilidad arquitectónica. Su fuerte sistema de tipos estático detectó inconsistencias en tiempo de compilación, y la estricta encapsulación de la mutabilidad (lectura de archivos y menús) dentro de la mónada IO garantizó que el procesamiento y optimización (QuickSort) de las rutas se mantuviera matemáticamente puro, predecible y libre de efectos secundarios.

En síntesis, Prolog demostró ser la herramienta más natural y ágil para prototipar el modelo de red y declarar las restricciones complejas gracias a su búsqueda implícita. En contraste, Haskell ofreció una base de ingeniería mucho más robusta; la separación de efectos y su tipado fuerte lo convierten en una solución algorítmica más escalable, segura y mantenible para gestionar el crecimiento de redes de transporte a largo plazo.